Real-time Target Speaker Extraction

Grant Booysen

25849646@sun.ac.za

Thesis presented in partial fulfilment of the requirements for the degree of

MSc in Machine Learning & Artificial Intelligence

in the Faculty of Science at Stellenbosch University

Supervisors: DJ Swanevelder and David Botha (SmartOperator)

November 2026

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Grant Booysen	**November 2026**
Signature	Date

Abstract

Acknowledgements

Contents

Nomenclature

Abbreviations

ASR	Automatic speech recognition
BSRNN	Band-split recurrent neural network
DNSMOS	Deep noise suppression mean opinion score
ICR	Interferer content rate
LCF-WER	Live-model content fidelity word error rate
LSTM	Long short-term memory
NRR	Non-response rate
SAR	Signal-to-artifacts ratio
SIR	Signal-to-interference ratio
SI-SDR	Scale-invariant signal-to-distortion ratio
STFT	Short-time Fourier transform
TF Map	Time-Frequency Map
TSE	Target speaker extraction
WER	Word error rate

Symbols

Chapter 1

Introduction

Chapter 2

Literature Review

2.1. Band-split Recurrent Neural Network (BSRNN)

Luo and Yu [1] propose the BSRNN model as a frequency-domain separation model that is designed for high sample rate audio, in which the input audio is transformed from its waveform representation into a time-frequency representation using the Short-Time Fourier Transform (STFT). The model then divides the input spectrum into multiple frequency bands that are modelled independently. The original use case for the BSRNN model is music source separation, but in this work, the model is adapted to allow for the extraction of a target speaker from a mixture of speech signals. The development from music source separation to the method of the target speaker extraction (TSE) approach that this work follows came from the following lineage: *(i)* music source separation was adapted to personalised speech enhancement (PSE) by Yu et al. [2]. The authors proposed target speaker extraction in the PSE task by appending a target speaker enrollment module that was designed to suppress any interference. PSE can be summarised as enrollment-conditioned extraction. *(ii)* Zhang et al. followed this framework and proposed a new module for the enrollment conditioning module, namely the Time-Frequency Map (TF Map) [3].

Chapter 3

Methodology

3.1. Baseline model architecture

3.1.1. Short-time Fourier transform

When speaking, the audio contains energy. The energy is not evenly distributed across frequencies, and in normal speech it is not evenly distributed across time either. The energy is concentrated in particular frequency bands by the unique resonances of the speaker's vocal tract, and is also concentrated in time by syllables, words and phrases. Audio models therefore want to leverage that structure, and require the input to be presented in a representation that allows for those concentrations to be observed. The waveform itself cannot represent that structure as it is a one-dimensional signal and is not aligned with the human perception of sound.

The short-time Fourier transform (STFT) produces that representation. The waveform is cut into short overlapping segments. These segments are used to compute the Fourier transform, which is used to decompose the signal into its frequency components. These overlapping segments are called frames, and the idea is to pass each of these segments through the Fourier transform by first multiplying the segment by a window function. The window function, which is typically a Hann window, tapers at the ends such that when multiplied, silences the ends of the segment to isolate the middle. After this a discrete Fourier transform is taken of each segment. The result is one complex number per frequency bin per segment: for a real input x, the coefficient at frequency bin f and frame t is

$$X(f,t) = \sum_{n=0}^{N-1} x[tH+n]\, w[n]\, e^{-j2\pi f n/N}, \tag{3.1}$$

where j is the imaginary unit ($j = \sqrt{-1}$), N is the window length in samples, H is the hop by which successive windows advance, and $w[n]$ is the window function. The hop is what makes the segments overlap: consecutive frames share $N - H$ samples, so a feature that falls near the edge of one frame sits comfortably inside the next. For the implementation in this work, a window size of $N = 512$ is selected, with a hop of $H = 128$ samples at $16\,\mathrm{kHz}$ gives a $32\,\mathrm{ms}$ window shifting by $8\,\mathrm{ms}$. These are the values Yu et al. [2] specify for their $16\,\mathrm{kHz}$

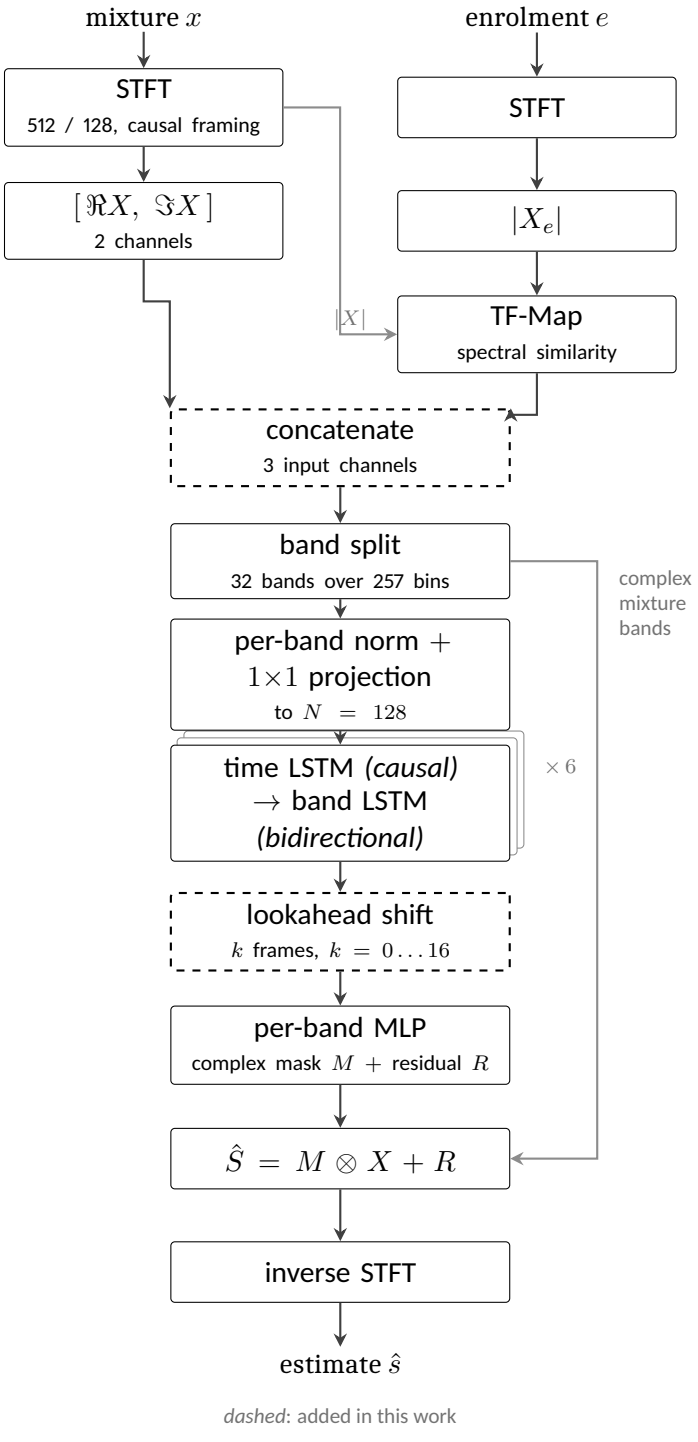

Figure 3.1: The extractor as implemented. Band splitting and the interleaved band-level and sequence-level modelling follow [1]; the mask-plus-residual estimator follows [2]; TF-Map conditioning follows [3]. The causal time path, the lookahead shift and the third input channel are this work's.

model. Secondly given the constraint of the streaming budget of $200\,\mathrm{ms}$ to $300\,\mathrm{ms}$, the framing consumes well under a quarter of that budget, and the remaining headroom allows for budget to be spent on the lookahead of Section 3.1.5. Finally, the approach in this work forms the frames without centring creating the causal framing. The reason for this is that centring each frame on its own timestamp makes frame t consume $N/2 = 256$ samples – $16\,\mathrm{ms}$ – of audio recorded *after* t. Since the objective is to create an online, streaming system and thus the model should not depend on future input. To account for the beginning samples of the audio, the signal is padded by $N - H$ samples at the start, which is a kind of cold start of a live system whose input buffer begins empty.

Each coefficient carries two quantities. Its magnitude $|X(f,t)|$ is how much energy the signal has in that frequency band at that time, and its phase $\angle X(f,t)$ is where in repetition of the waveform the signal is at that time. The magnitude is what the model uses to determine which parts of the mixture belong to the target speaker, and the phase is what the model uses to reconstruct the waveform from the modified magnitude. The complex spectrogram is kept separately, since the mask of Section 3.1.6 is applied to it rather than to the network's internal features.

3.1.2. Band plan

The spectrogram of Section 3.1.1 has 257 outputted frequency bins, each covering an equal slice of $31.25\,\mathrm{Hz}$. Treating all 257 alike would waste the model's capacity, because speech does not spread itself evenly across them. Most of the energy, and nearly all of the structure that distinguishes one voice from another, sits at the lower frequencies: the pitch of a voice and the resonances that give a vowel its identity all fall below about $4\,\mathrm{kHz}$, and energy falls away steadily above that. The bins near the top carry far less information and thus would be wasteful to model with the same level of detail as the bins near the bottom.

A band plan is the decision of how to group those bins. Bins are gathered into contiguous groups, called bands, and each band is then handled separately by the network: it gets its own normalisation and its own projection, so a band is compared against itself over time rather than against the loud low-frequency bands next to it. Grouping few bins per band keeps fine detail. Secondly grouping many bins into one band summarises coarsely but costs far less. A band plan therefore decides where the model's resolution is spent.

The plan in this work is 32 bands over the 257 bins: fifteen bands of $100\,\mathrm{Hz}$ covering $0\,\mathrm{kHz}$ to $1.5\,\mathrm{kHz}$, ten of $200\,\mathrm{Hz}$ covering $1.5\,\mathrm{kHz}$ to $3.5\,\mathrm{kHz}$, five of $500\,\mathrm{Hz}$ covering $3.5\,\mathrm{kHz}$ to $6\,\mathrm{kHz}$, and the remaining $6\,\mathrm{kHz}$ to $8\,\mathrm{kHz}$ as one band. Figure 3.2 shows it against a real mixture in the training set. The effect is a factor of twenty between the finest and coarsest resolution. This plan follows the implementation from Zhang et al. [3] for their $16\,\mathrm{kHz}$ target speaker extraction model and used by the challenge baseline built on it, but no reason is given there for these particular boundaries, and no explanation is given in previous papers. The original band

splitting was designed for music [1], where the sources are instruments rather than voices. It is adopted here as the baseline so that this work's results are comparable with that lineage, and it is recorded as an unjustified inheritance rather than a motivated choice.

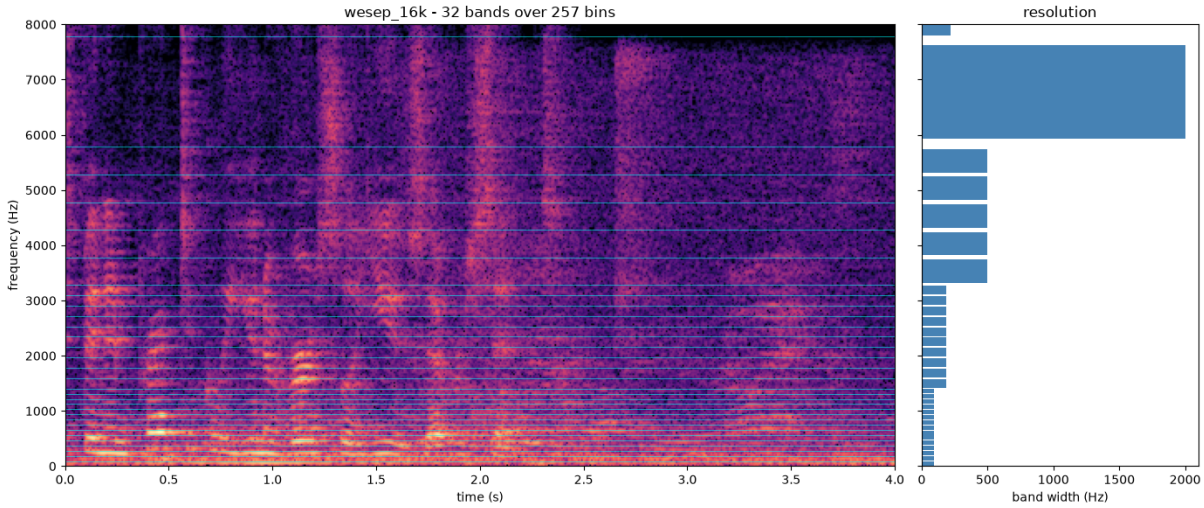

Figure 3.2: The band plan used here, shown against a four-second mixture from the training set. Left: the band boundaries drawn over the spectrogram, closely spaced at the bottom where the speech structure is and widening upwards. Right: the width of each band in hertz. The lowest fifteen bands are $100\,\mathrm{Hz}$ wide; the single highest spans $2\,\mathrm{kHz}$. This band plan follows the one used by Zhang et al. [3] for their $16\,\mathrm{kHz}$ model.

[GRANT: TODO - Possible experiment] *Because the plan is a free parameter that nobody has justified, it is also implemented as a configurable table rather than fixed in code, with alternative plans available for comparison.*

Why the spectrum stops at $8\,\mathrm{kHz}$. The audio in this work is sampled at $16\,\mathrm{kHz}$, meaning 16,000 amplitude measurements per second. When measuring a wave you need at least two measurements per cycle, one for the peak and one for the trough of the wave. Therefore the fastest possible wave that $16\,\mathrm{kHz}$ sampling can represent will complete 8,000 cycles per second. Anything faster cannot be distinguished from a slower wave passing through the same measured points. Therefore, the largest possible range that can be represented by the 257 bins have to span $0\,\mathrm{kHz}$ to $8\,\mathrm{kHz}$ rather than $0\,\mathrm{kHz}$ to $16\,\mathrm{kHz}$.

3.1.3. Per-band normalisation

The bands defined by Section 3.1.2 are not comparable with one another. Speech energy falls away by roughly $6\,\mathrm{dB}$ per octave, so the lowest bands carry orders of magnitude more energy than the highest. Therefore, if all 257 bins were scaled together, there would be a severe imbalance where the high bands would be underrepresented. This would then hurt the performance of the model as the high bands would be diluted and would contribute almost nothing to the gradient, and the model would in effect only learn to separate the bottom of

the spectrum. Giving each band its own scaling is what prevents that. A band at $6\,\mathrm{kHz}$ is then judged against its own typical level rather than against the bands beneath it.

This module does two things to each band independently, following [2]. First it normalises the band, using a layer normalisation applied across the channels of that band. Secondly, because the bands must be the same size before they can be stacked together, a projection is required to map the band to a fixed width. Since there are varying sized and independent bands, each band has its own projection, which is a learnable linear layer that maps the band to a fixed width of $N = 128$ values. Every band is given the same structure but its own weights, so nothing is shared between bands and each is free to learn a representation suited to its own frequency range. The result is a stack of 32 bands, each with 128 features, which is then passed to the next module.

Normalisation is applied across the values of a band *within a single frame*. Each frame is therefore normalised using only itself. This is a deliberate design choice that opposes Yu et al. [2], who use batch normalisation for their online configuration. The opposition is for the following reasons: *(i)* although batch normalisation will work for online models as indicated by Yu et al. and is causal, there are two different operations of batch normalisation: training and inference. During training, the mean and variance are computed from the current batch, but during inference, these statistics are computed from the entire training set. The function that is optimised during training is therefore not the function that is deployed. *(ii)* That gap is unusually wide here. A batch is twelve four-second crops drawn at random from a corpus spanning a range of reverberation times, signal-to-interference ratios and noise levels, and in which roughly three crops in ten contain no target speech at all. The statistics of a batch therefore depend on what was drawn, and a silent crop normalised alongside loud ones is not normalised as it would be at inference. Because normalisation is per band, the effect is worst in the high bands, where the energy is smallest and the variation between batches is largest relative to the signal. *(iii)* Single frame normalisation carries no state and is unaffected by the amount of audio processed, so a four-second training chunk is normalised exactly as a stream that has been running for a minute.

3.1.4. Residual LSTM block

The idea of the residual LSTM block is to model the sequential structure of speech such that the model can leverage patterns that unfold over time. There are two kinds of sequential structures that are modelled here. The first is the sequence over time, which is the natural order of speech. The second is the sequence over frequency. Think of the time being unidirectional, as the speech only makes sense to unfold forward in time, but the frequency is bidirectional, as all of the bands of a frame are available at the same instant and can be traversed in either direction. The block is used for both kinds of modelling, with the same structure but different parameters.

The recurrent layer is what carries information forward where this work leverages a long

short-term memory layer, or LSTM. This is a recurrent layer with an internal mechanism for deciding what to keep in that summary and what to discard, which is what allows it to retain information over hundreds of frames rather than only the last few. This structure also suits the deployment target directly. At inference time, since the model is running in a streaming fashion, the LSTM's internal state is preserved between frames and information can be carried forward indefinitely. The LSTM is therefore a natural fit for the task, and it is used in the same way for both the time and frequency sequences.

The block is made of three steps applied in order, following [1]. The input is first normalised, then passed through the LSTM, and the LSTM's output is then mapped back to the input's width by a learned projection. The final addition is a residual connection and is inspired by the original ResNet [4] architecture. This allows the gradient an unobstructed path back through the computation stack of all the blocks, which matters because six of these blocks are stacked in Section 3.1.5. This is important as the LSTM is a deep recurrent layer, and the gradient can vanish or explode as it passes through the LSTM's internal state. The residual connection allows the gradient to bypass the LSTM entirely, which is what allows the model to train when the gradient vanishes within the LSTM.

[GRANT: TODO] - Still might need to increase the size of the model so this is not final!!!

What is used here. The LSTM has a single layer with a width of 192, taken from Yu et al. [2], whose $16\,\mathrm{kHz}$ model specifies that width. This is deliberately narrower than the reference implementation of the same architecture, which uses twice the feature width, or 256. Because the LSTM's width differs from the 128 features entering the block, the projection is not optional bookkeeping: it is what makes the addition dimensionally possible at all.

The block is written to be indifferent to what its sequence axis represents. It receives a batch of sequences and processes them; whether those sequences run along time or along frequency is decided by the caller. This is what allows the same block to be used for both kinds of modelling in Section 3.1.5, with one critical difference between the two uses. Along time the LSTM runs forward only, because a backward pass over time reads frames that have not yet arrived. Along frequency it runs in both directions, which costs nothing in latency, since all the bands of a frame are available at the same instant.

3.1.5. Band and sequence model

The band and sequence model is the implementation of the residual LSTM block of Section 3.1.4 in a stack of six blocks, using the input from Section 3.1.3. After the per-band normalisation and projection, there are 32 bands of 128 features each. The model is now tasked for the two kinds of sequential modelling with the residual LSTM blocks as previously mentioned: time and frequency.

Each of these blocks is designed to make two passes. The design is to first make a pass

over the time dimension (unidirectional), and then make a pass over the frequency dimension (bidirectional). This is considered as follows:

- **Time**: The recurrence goes forward only, as a backward pass would assume knowledge of future frames that have not yet arrived.
- **Frequency**: The recurrence goes over the bands of a frame in both directions. This pass is what lets the bands inform one another: a voice lays down a harmonic pattern across many bands at the same instant, so whether a given band belongs to the target is often not decidable from that band alone. Frequency itself is not a causal dimension. This can be reasoned by the fact that all the bands of a frame are all available simultaneously so the bidirectional recurrence does not introduce any latency.

Pairing the two passes in a single block is the design of Luo and Yu [1], who motivate it by analogy with dual-path recurrent networks for speech separation rather than by an argument specific to bands. Stacking six such blocks follows Yu et al. [2], where their $16\,\mathrm{kHz}$ model uses that specific number of blocks. The effect of stacking the blocks is to allow for the accumulation of evidence over longer time spans. The evidence can then be shared across bands of the next block and so forth.

Lookahead. The model is set to allow for a lookahead of k frames, ranging from 0 to 16, which costs k hops of additional latency, $8k$ milliseconds. The REAL-TSE challenge, an IEEE SLT 2026 satellite challenge [5] counts lookahead frames as part of the latency budget and does not consider it as a parameter that will break the online requirement. It is rather an expense for the overall latency budget. CARTSE [6] uses no lookahead frames at all, so its algorithmic latency reduces to the window minus the hop. The challenge overview reports no comparison of lookahead against the overall quality of the TSE task, so whether it helps is an open question rather and **is treated as an experimental question**. ¡- [TODO - Grant: Need to test this lookahead or remove if not applicable]

The main idea behind using lookahead is that it can be reasoned that often evidence for whether a frame belongs to the target speaker might arrive late. If the model waits slightly to see what comes next for a brief moment, it does not need to commit to a decision immediately. At $k = 0$ the model has to commit to a decision.

[GRANT TODO] - NEED TO TEST THIS LOOKAHEAD *The baseline is trained at $k = 0$ so that the fully causal model is what any larger setting is measured against. The additional delay is treated as an experimental question in Chapter 4 rather than assumed here.*

It is implemented by pairing each frame with a later summary of the sequence. The model still processes every frame as it arrives. What changes is which of the stack's output frames the estimation module will read. To produce the mask for frame t it reads the state the network produced at frame $t + k$ rather than its state for t. Since the recurrence for time only goes

forwards, the model will consider everything that was heard up to frame $t+k$ when considering the estimation for frame t.

3.1.6. Estimation module

The estimation module is responsible for producing an output spectrogram that contains the estimated target speaker's speech. This is done by creating an output network per band to reconstruct the target speaker's speech from the features produced by the band and sequence model. However, there are two methods to do this: either to predict the output spectrogram directly, or to predict a mask that is applied to the mixture. The latter is the approach taken here, following Yu et al. [2].

Yu et al. realised that the mixture audio itself already contains the target's speech, and thus it would be more efficient to mask the mixture rather than predict the output spectrogram directly. This mask is a number for each time-frequency bin, which the mixture is multiplied by.

A mask alone is not enough however as multiplication can only ever scale what a bin already holds. In the cases where the interferer completely dominates a bin no multiplier recovers the target as the multiplication operation would not be able to scale the drowned out target. To solve this, the estimation module makes use of a residual spectrogram and is predicted directly and appended. Yu et al. [2] introduce it for exactly this reason, to absorb what the mask alone produces as artefacts. It is useful to not bound this mask and is what the original implementation encouraged. The estimate for a band is then

$$\hat{S} = M \otimes X + R, \tag{3.2}$$

where $\otimes$ is the complex product, taken bin by bin. The X that the mask multiplies is the original complex mixture and is taken as is from the band split and not from the output of the model's features. The features are used only to *decide* how to mask the input. The conditioning feature of Section 3.1.7 therefore never enters the signal path either.

Each band has its own two-layer output network: a normalisation, then a projection to a width of 384 with a $\tanh$ activation. Finally we have two output layers reading from it, one producing the mask and one the residual. The width of 384 is the value Yu et al. [2] specify.

The mask output layer has to output four pairs of real and imaginary parts for each frequency bin. Two of them are taken as the mask values, and the other two are passed through a sigmoid, giving a number between zero and one that each mask value is then multiplied by. This arrangement is a gated linear unit [7], and it is what both source papers specify for this layer [1, 2].

3.1.7. Time-frequency map

The TF Map is the enrollment-conditioning module to turn normal speech enhancement into the TSE task. The idea behind this module is for the model to take an enrollment of the target speaker, and use the unique audio signature of that speaker to condition the model to extract only that speaker from the mixture. The general process includes processing the enrollment clip through the STFT module, extracting the magnitude of the enrollment, and then breaking up the spectrogram into basis vectors. These basis vectors are then compared to the mixture's spectrogram using cosine similarity.

Concretely, let $|X| \in \mathbb{R}^{F\times T}$ be the magnitude spectrogram of the mixture and $|X_e| \in \mathbb{R}^{F\times T_e}$ that of the enrollment, and write $|X_t|$ and $|X_{e,j}|$ for one frame of each. The module proceeds in three steps. *First*, every frame of both spectrograms is scaled to unit length,

$$\tilde{x}_t = \frac{|X_t|}{\big\||X_t|\big\|}, \qquad \tilde{e}_j = \frac{|X_{e,j}|}{\big\||X_{e,j}|\big\|}. \tag{3.3}$$

Second, each mixture frame is compared against every enrollment frame by cosine similarity. Because (3.3) has already removed the length of each frame, the cosine of the angle between two frames is simply their inner product:

$$S_{t,j} \;=\; \cos\theta_{t,j} \;=\; \frac{\big\langle |X_t|,\ |X_{e,j}| \big\rangle}{\big\||X_t|\big\|\,\big\||X_{e,j}|\big\|} \;=\; \big\langle \tilde{x}_t,\ \tilde{e}_j \big\rangle. \tag{3.4}$$

This is what makes the comparison of spectral *shape* rather than of loudness: a quiet frame and a loud frame with the same spectral pattern are identical under (3.4). Note also that magnitudes are non-negative, so $S_{t,j} \in [0, 1]$ for every pair.

Third, the similarities of a mixture frame are turned into weights over the enrollment, and those weights are used to blend the enrollment frames,

$$H_{t,j} = \underset{j}{\mathrm{softmax}}\,\big(S_{t,j}\big), \qquad M_t = \sum_{j=1}^{T_e} H_{t,j}\,\tilde{e}_j, \tag{3.5}$$

following [3]. The softmax runs along the enrollment index j, so each row of H sums to one and describes how to blend the enrollment frames for one instant of the mixture. M_t is a guessed spectrum: for each mixture frame, a weighted average of all the enrollment frames, with the weights given by H. It is a rough estimate of the shape of the target speaker's spectrum at that instant.

M_t describes only that shape. It is built entirely from enrollment frames but on its own it says nothing about whether the target is speaking. Note also that the mixture's magnitude has not been used at any point so far: (3.3) removed it before the comparison in (3.4) was made. The final step is to address the energy component. The estimate is scaled to unit length and

the mixture's spectrum is projected onto it which measures how much of the mixture's actual energy lies in the pattern the enrollment predicted:

$$f_t = \left\langle\, |X_t|,\ u_t \,\right\rangle u_t, \qquad u_t = \frac{M_t}{\|M_t\|}, \tag{3.6}$$

The feature therefore points in the direction the enrollment predicts and has the magnitude of however much of the mixture actually lies in that direction. Where the mixture at that instant looks like the target, it is large otherwise small. It is this that the model learns to use to identify which parts of the mixture belong to the target speaker.

Where it enters and what it costs. The result is a quantity of the same shape as the spectrogram itself, one value per frequency bin per frame. It is therefore attached as a third input channel alongside the real and imaginary parts, before the band split, so that it is divided into bands and projected by the same machinery as the rest of the input. Nothing else in the architecture changes. Because the extra channel only widens the input of the per-band projections, the entire conditioning path costs roughly $33\,000$ parameters, under half a percent of the model, and it adds no learned parameters of its own at all: (3.3) through (3.6) contain nothing to train.

3.1.8. Objective function

The objective has to express two different requirements, because the training data contains two different situations. Where the target speaks, the output should match the target signal. Where the target is silent as the result of either the trial contains no target speech at all, or because the four-second crop happened to land in a gap between the target's utterances, there is no signal to match, and the correct output is silence. A single loss cannot serve both use cases and so the objective is therefore split, and each crop is scored by the branch that applies to it.

Throughout, x is the mixture, s the target reference and $\hat{s}$ the model's output, all in the time domain over one training crop.

Target present. The first term measures how much of the output is the target and how much is everything else, following [6]. Importantly, the loss needs to be invariable to the overall volume as if the $\hat{s}$ is of the correct shape, it should not be penalised for being loud or quiet. Therefore the first step is to find the single gain that best explains the output as a scaled copy of the target, which will be calculated as a projection,

$$\alpha = \frac{\langle \hat{s},\ s \rangle}{\|s\|^2}, \qquad s_{\text{proj}} = \alpha s, \tag{3.7}$$

and then comparing the energy of that scaled target against the energy of whatever the output contains beyond it. The loss is modelled after the Scale-Invariant Signal-to-Distortion Ratio

(SI-SDR). The loss is therefore,

$$L_{\text{pres}} = -10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|\hat{s} - s_{\text{proj}}\|^2 + \tau \|s_{\text{proj}}\|^2}. \tag{3.8}$$

Lower is better. The loss is secondly in terms of decibels (dB), which is where the logarithm comes from. The τ term in the denominator is a floor. Without it a perfect output would score $-\infty$, and the gradient would keep rewarding refinement of crops that are already essentially solved. With $\tau = 10^{-3}$ the best attainable score is $-30\,\text{dB}$. This constant follows the suggestion from CARTSE [6]. The reasoning is that an output whose residual error already sits $30\,\text{dB}$ below the target is close enough to be treated as solved, so it would be wasteful to keep spending gradient on it. Two properties of this measure should be stated plainly, because they are what the second term exists to compensate for. The first is that everything the output contains beyond the target is collected into one denominator. Residual interfering speech, residual background noise and artefacts the model itself introduced all count identically per unit of energy, although they are not equally damaging: a fragment of the interferer left in the output can be transcribed downstream as words the target never said, whereas a diffuse artefact of the same energy usually cannot. The measure has no way to express that difference. The second is that it is an energy ratio summed over the whole crop, so it is dominated by whatever is loudest in it. Speech energy sits overwhelmingly in the low frequencies and in vowels, which means a model can score well on (3.8) while reconstructing fricatives and the upper spectrum poorly. Those are the parts that carry consonant identity, and therefore much of what makes speech intelligible rather than merely present.

Multi-resolution spectral term. Equation (3.8) is a single number computed over the whole waveform, which makes it ignore the location of errors in the spectrum. Since it cannot express *where* in the spectrum the error lies, a model that reconstructs the loud low bands well and the quiet high bands poorly is barely penalised and secondly since being scale-invariant, it does not constrain the output's absolute loudness at all. The idea is to use a loss that considers multiple resolutions of s and $\hat{s}$. By considering different granularity of the spectrogram, the model is encouraged to reconstruct both the low and high frequency bands well. A spectral term allows for this following [2]:

$$L_{\text{MR}} = \frac{1}{I} \sum_{i=1}^{I} \left(\left\| |S_i|^p - |\hat{S}_i|^p \right\|_1 + \left\| \Re S_i - \Re \hat{S}_i \right\|_1 + \left\| \Im S_i - \Im \hat{S}_i \right\|_1 \right), \tag{3.9}$$

where S_i and $\hat{S}_i$ are the spectrograms of the target and the output under the ith of I analysis window lengths, and each norm is averaged over its time-frequency coefficients.

Three choices inside (3.9) are worth stating. The exponent $p = 0.3$ compresses magnitudes, which will highlight the quieter components relative to loud ones so that an error in a low-energy region is not diluted just because it is quite. Without the compression the term would

be dominated by the same loud low bands that already dominate (3.8). The real and imaginary terms are left uncompressed and they are what make the term sensitive to phase: a magnitude-only loss would be indifferent to the phase that the complex mask of Section 3.1.6 is free to change. Finally the term is evaluated at several window lengths rather than one, because a single analysis window can hide errors at its own scale.

The window lengths used are 8, 16, 32 and $64\,\mathrm{ms}$. This is with respect to the model's own $32\,\mathrm{ms}$ framing. The model builds its output by masking $32\,\mathrm{ms}$ frames and overlapping them, so the places that most errors would occur can be expected to exist at frame boundaries. The $8\,\mathrm{ms}$ and $16\,\mathrm{ms}$ windows resolve this and the $64\,\mathrm{ms}$ window catches harmonic structure that the short ones blur.

Target absent. When there is no target speaker present in the mixture audio, the model should output silence and suppress all other noise. The model should therefore penalise any output that is not silent. This is modelled after a modified version of the absolute level loss from CARTSE [6], where in this works the loss now contains the denominator term.

$$L_{\text{abs}} = 10 \log_{10} \frac{\|\hat{s}\|^2 + \tau \|x\|^2}{\|x\|^2}. \tag{3.10}$$

Including the denominator term makes the loss relative to the input, which is directly readable: $0\,\mathrm{dB}$ means the output is as loud as the mixture, that is, the model did nothing; $-10\,\mathrm{dB}$ means the interfering speech was suppressed by $10\,\mathrm{dB}$; and the same τ floors the term at $-30\,\mathrm{dB}$, beyond which the output counts as silent. A positive value means the model amplified the mixture, which is a direct failure. The inclusion of the denominator also allows for scale invariance, which puts the loss in the same units as the loss for the target present case.

Combining the two. Each term is averaged over the crops it applies to, and the two averages are weighted against one another:

$$L = (1 - w) \underset{\text{target present}}{\text{mean}} \Big[L_{\text{pres}} + w_{\text{m}} L_{\text{MR}} \Big] + w \underset{\text{target absent}}{\text{mean}} \Big[L_{\text{abs}} \Big]. \tag{3.11}$$

Averaging within each branch rather than over the batch as a whole matters more than it appears to. Under a single batch-wide mean, the share of the gradient belonging to the silent crops would be whatever fraction of the batch happened to be silent, so the balance between the two requirements would fluctuate from step to step on the luck of the draw. Under (3.11) that balance is fixed by w.

Which branch a crop belongs to is decided from the audio of the cropped target itself, not from the trial's label. A trial in which the target does speak can still yield a crop containing no target speech, and this occurs in $5.8\,\%$ of crops. Routing such a crop to the present branch would put an all-zero reference into (3.7), where α becomes $0/0$. the implementation asserts against this rather than relying on the labels agreeing.

Weights. The weight w on the silent branch is derived rather than chosen. Silent crops make up a measured 0.297 of the training crops, and the objective is intended to weight silence at twice its frequency in the data, following [6]. Carrying that through gives $w = 0.458$, which reproduces the same focus on silence as the source objective, but using the actual frequency of silence in this project's training data rather than the source's.

The weight w_{m} on the spectral term is derived from the target present and absent losses. The reason for this term is equations (3.8) and (3.9) are in different units. L_{pres} is in units with respect to a ratio in decibels, and L_{MR} is the mean absolute difference between compressed magnitudes. Therefore the weighting term w_{m} will reconcile the two terms to be on the same scale. This is done by evaluating on the output that does nothing at all, $\hat{s} = x$, over 300 training crops which will simulate the scenario for the average losses for an untrained model. The median values for L_{pres} and L_{MR} are -5.91 and 0.184 respectively. The aim is for the spectral term (Multi-Resolution) to be roughly $30\,\%$ of the present branch at the start of training. The calculation then becomes

$$w_{\mathrm{m}} = \frac{0.3 \times |L_{\mathrm{pres}}|}{L_{\mathrm{MR}}} = \frac{0.3 \times 5.9088}{0.1842} = 9.62$$

Differences with the source objective. The present and absent terms are inpsired from the CARTSE system [6], the online-track submission whose task most closely matches this one, but they are slightly modified. Table 3.1 lists the differences. The first is the most consequential: as published, the floor in (3.8) is taken on $\|s\|^2$ rather than on $\|s_{\mathrm{proj}}\|^2$, which breaks the scale invariance the term is built on. With that form, an output of the correct shape scaled by a gain g scores better the louder it is made and it is not bounded. Flooring on the projection instead makes numerator and floor scale together, so the two cancel and the score is flat with respect to gain.

3.2. Metrics

This section describes the evaluation metrics used to assess the performance of a trained target speaker extraction model. These metrics are where the main contribution of the work lies. The particular use case of the TSE model designed in this work is to extract target speech, from a mixture of target and interfering speech, where the model has to be setup to handle the following cases:

- **Case 1:** Target speech is present, interfering speech is present, and background noise is present.
  - **Case 1a:** Interfering speaker interrupts the target speaker, i.e. the target speaker is cut off by the interfering speaker.

Table 3.1: Departures from the objective of [6] as published. Each is recorded with its reason in the project's decision log.

	As published	As used here
1. Floor in (3.8)	On $\tau\|s\|^2$, which is not scale-invariant	On $\tau\|s_{\text{proj}}\|^2$, so that gain cancels
2. Scale of (3.10)	Unnormalised, so the value shifts with input loudness	Divided by $\|x\|^2$, so $0\,\text{dB}$ means "did nothing"
3. Averaging over the batch	One mean over all crops in the batch	A mean within each branch, so the balance is set by w alone
4. Number of weights	A separate weight on the silent term	Folded into w; under branch-wise means the two are one degree of freedom
5. Silence weight	Twice the absent rate of the source corpus	$w = 0.458$, the same emphasis at this project's measured 0.297
6. Windows in (3.9)	All at or above the model's frame length	8, 16, 32 and $64\,\text{ms}$, slightly lower and higher than the model's $32\,\text{ms}$ frame length

  - **Case 1b:** Interfering speaker speaks over the target speaker, i.e. the target speaker is still speaking while the interfering speaker is also speaking.
  - **Case 1c:** Turn-taking, i.e. the target speaker and interfering speaker take turns speaking, with no overlap and space between them.
  - **Case 1d:** Target speaker louder/softer than interfering speaker, and speaks for longer/shorter than interfering speaker.

- **Case 2:** Target speech is present, interfering speech is absent, and background noise is present.
- **Case 3:** Target speech is absent, interfering speech is present, and background noise is present.
- **Case 4:** Target speech is absent, interfering speech is absent, and background noise is present.

Since the main task is intelligibility of the target speaker and not necessarily the *quality* of the target speaker, the metrics that are designed in this section aim to reflect that objective. The metrics are designed for the purposes of using the BSRNN model in conjunction with a live speech-to-speech model, such as Gemini Live [8]. Such a model is not a transcriber: it consumes audio through a learned encoder trained over a far wider distribution than read speech, and it is prompted rather than decoded. The metrics are therefore meant to show how well the model will translate as part of a live conversational system, and hence are designed to be used in conjunction with that system.

3.2.1. Trial definitions

Each trial passed through the model at evaluation produces a 4-tuple $(y_x, y_{\hat{s}}, y_s, y_d)$ where y_x is the input mixture speech transcription, $y_{\hat{s}}$ is the estimated target speech transcription, y_s is the true target speech transcription, and y_d is the interferer speech transcription. The subscripts follow the audio notation of Section 3.1.8: x is the mixture, s the target reference and $\hat{s}$ the model output, with d denoting the interfering speech and $y_{(\cdot)}$ the transcription of a signal. These transcriptions are created in two ways in this project: (1) by using an automatic speech recognition (ASR) system [9, 10] to transcribe the audio to act as a baseline transcriber, and (2) by making use of a live speech-to-speech model that accepts the audio and outputs a transcription which is prompted with a standard prompt to transcribe the audio found in Appendix A.1. The tuple therefore does not represent necessarily the true transcriptions, but rather what the model/transcriber produces. To avoid confusion, the true transcriptions as found in LibriSpeech will be denoted with a star, e.g. $y_s^\star$

3.2.2. Text normalisation

To standardise the transcriptions created by the different methods, a text normalisation process is applied to the audio transcriptions. Specifically, using the `whisper` library [11], the `EnglishTextNormaliser` package, seen in Appendix C of [10], is used to output the transcriptions into lower case, remove punctuation, expand contractions, standardise the spelling of numbers, and remove any non-alphanumeric characters. This is done to ensure that the transcriptions are comparable across different methods of transcription.

3.2.3. Live-model Content Fidelity Word Error Rate

The Live-model Content Fidelity Word Error Rate (LCF-WER) is a metric based off of the arithmetic of the standard Word Error Rate (WER) metric, but applied to a live-model transcription of the audio instead of a standard transcription. It is explicitly labelled as such as the standard WER metric is referenced in the training procedure, and is not associated with the live-model transcription. The standard WER metric is defined as follows:

$$\text{WER} = \frac{S + D + I}{N} \tag{3.12}$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference. It is essentially a minimum edit-distance formula, where the number of edits required to change a reference text into the proposed text is divided by the number of words in the reference text. However, there is one flaw in this approach that is required for our metrics to highlight: the broad classes of substitutions, deletions, and insertions do not demonstrate the nuance of how these errors can

occur. The LCF-WER is used to represent that fact that the classes of errors are specific to live model transcription.

3.2.4. Interferer Content Rate

The Interferer Content Rate (ICR) is the first metric used to highlight one particular nuance of the LCF-WER metric: a substitution can occur when the model hallucinates a word that is not present in the reference, or the substitution can occur with the model hearing the right word, but from the wrong speaker. The idea for the ICR metric is to highlight the latter case, where the model has heard the correct word, but from the wrong speaker. This will help determine if our model is following speakers, or making up words that are not present in the reference.

Content Words. The ICR metric needs to identify which words are likely to be words that are not filler words that a model can easily hallucinate. Let $\mathcal{C}(\cdot)$ map a normalised transcription to the *set* of content words it contains,

$$\mathcal{C}(y) = \Big\{ w \in \text{words}(y) \ : \ w \notin \mathcal{S} \Big\}, \tag{3.13}$$

where $\mathcal{S}$ is a fixed stopword list, taken from the NLTK English stopword corpus [12]. Function words are removed because almost any two English sentences share them, so an overlap counted over these words would rather count as grammatical issues than content. These content words will be used to make interferer-exclusive bag of words of which will indicate which words the model picks up from the interfering speaker.

Interferer-exclusive content. Firstly, if both the target and interfer say the same word, it is not possible to know which speaker the model heard, thus these words are not considered unique to the interferer. Let $\mathcal{D}$ be the unique content words of the interferer that are not present in the target,

$$\mathcal{D} = \mathcal{C}(y_d) \setminus \mathcal{C}(y_s), \tag{3.14}$$

and the words that will be considered as leaked from the interferer are the intersection of the model output, $y_{\hat{s}}$ and the interferer-exclusive content words, $\mathcal{D}$,

$$\mathcal{L}(y_{\hat{s}}) = \mathcal{C}(y_{\hat{s}}) \cap \mathcal{D}. \tag{3.15}$$

Write $\ell = |\mathcal{L}(y_{\hat{s}})|$ for the number of words leaked on a trial and $a = |\mathcal{D}|$ for the number of possible unique words to leak. If a trial has $a = 0$, then the trial is not considered in the ICR metric, as there are no unique words to leak.

The contribution that this paper makes is to turn the ICR metric into a rate, as sentences are varying lengths, and thus the amount of leaked words changes with the length of the sentence. The rate secondly helps to answer the following situation: should the model that is finally

chosen leak fewer words frequently, or leak more words infrequently. The rating definition is aimed to answer that question, and is defined as follows:

The rate. The rate can be reported at several thresholds. For a set of trials $\mathcal{T}$, and a threshold k of leaked words in a trial, the ICR metric is defined as the fraction of trials in which the number of leaked words ℓ is greater than or equal to k,

$$\text{ICR@}k = \frac{\left|\{\, i \in \mathcal{T}_k \;:\; \ell_i \geq k \,\}\right|}{|\mathcal{T}_k|}, \qquad \mathcal{T}_k = \{\, i \in \mathcal{T} \;:\; a_i \geq k \,\}. \tag{3.16}$$

Lower percentages are better.

The eligible set $\mathcal{T}_k$ depends on k, and this is deliberate. If in a trial there is only one exclusive content word available, it is impossible to achieve ICR@2, so if we keep this trial in the denominator it would skew the results.

3.2.5. Non-Response Rate

3.2.6. Anchors

3.2.7. Reporting protocol

Chapter 4

Experiments

Chapter 5

Results

Chapter 6

Conclusion

Bibliography

[1] Y. Luo and J. Yu, "Music source separation with Band-Split RNN," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1893–1901, 2023.

[2] J. Yu, H. Chen, Y. Luo, R. Gu, and C. Weng, "High fidelity speech enhancement with Band-split RNN," in *Proc. Interspeech 2023*, 2023, pp. 2483–2487.

[3] K. Zhang, J. Li, S. Wang, Y. Wei, Y. Wang, Y. Wang, and H. Li, "Multi-level speaker representation for target speaker extraction," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[5] S. Wang, Z. Qian, K. Zhang, J. Han, Z. Liu, X. Yu, H. Li, M. Delcroix, K. Yu, L. Xie, M. Li, and H. Li, "SLT 2026 REAL-TSE challenge: Real-world target speaker extraction from conversational recordings," arXiv, Tech. Rep. arXiv:2607.15198, 2026.

[6] L. Li and S. Seki, "CARTSE submission to the REAL-TSE challenge track 1: Online target speaker extraction," CyberAgent, Inc., Tech. Rep., 2026, system description.

[7] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," in *Proc. 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 933–941.

[8] Google DeepMind, "Gemini Live API," https://ai.google.dev/gemini-api/docs/live, 2026, accessed: 2026-08-31. Exact model identifier to be recorded with each result.

[9] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," https://github.com/SYSTRAN/faster-whisper, 2026, version 1.2.1. Accessed: 2026-08-31.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518, also arXiv:2212.04356. The English text normaliser used in this work is defined in Appendix C.

[11] whisper-normalizer contributors, "whisper-normalizer: Standalone text normalisation from OpenAI Whisper," https://pypi.org/project/whisper-normalizer/, 2026, version 0.1.15. Implements the EnglishTextNormalizer of [10]. Accessed: 2026-08-31.

[12] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009, the English stopword list is taken from the NLTK stopwords corpus and snapshotted into this project's source tree so the list cannot change with a library upgrade.

[13] jiwer contributors, "jiwer: Evaluate automatic speech recognition," https://github.com/jitsi/jiwer, 2026, version 4.0.0. Accessed: 2026-08-31.

Appendix A

Evaluation protocol details

A.1. Prompt for live-model transcription

To be completed once the judge model is selected.

A.2. Pinned components of the measuring instrument

Component	Identity
Text normaliser	`whisper-normalizer` 0.1.15, `EnglishTextNormalizer` [10, 11]
Baseline transcriber	`faster-whisper` 1.2.1, `small.en`, int8 quantisation, greedy decoding [9, 10]
Word error rate	`jiwer` 4.0.0 [13]
Stopword list	`NLTK` English stopwords [12], snapshotted into the source tree so the list cannot change with a library upgrade
Judge model	*to be recorded*
Random seed	42

Table A.1: Components of the evaluation instrument, pinned by version. A change to any row invalidates comparison with numbers reported before it.